TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT
THÀNH PHỐ HỒ CHÍ MINH
KHOA ĐIỆN-ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ CÔNG NGHIỆP – Y SINH

ĐÁP ÁN ĐỀ THI CUỐI KỲ HỌC KỲ 1 - NĂM HỌC 2019-2020
Môn: **VI XỬ LÝ**
Mã môn học: MICR330363
Đề số/Mã đề: 01 Đề thi có: 01 trang
Thời gian: 90 phút.
Sinh viên chỉ được tham khảo một tờ giấy A4 viết tay

Câu 1: (3 điểm)

Xét một ứng dụng PIC16F887 (F_{OSC} = 8MHz, tùy chọn nguồn dao động trong hoặc ngoài) tạo xung PWM với các yêu cầu như sau:

- Xung PWM có chu kỳ T_{PWM} = 2ms xuất ra tại chân CCP1.
- Một nút nhấn thường hở (tên gọi ADJ, được nối đến RE0) để điều chỉnh giá trị trên một LED bảy đoạn (tên gọi LEVEL, được nối đến PORTD). Phạm vi giá trị LEVEL = [0, 1, …., 8, 9], tương ứng với 10 cấp chu kỳ công tác DUTY = [0%, 10%, …, 80%, 90%] của xung PWM. Mỗi lần nhấn ADJ, LEVEL tăng một đơn vị. Khi LEVEL = 9, nếu nhấn ADJ thì LEVEL ⇨ 0. Lúc mới bật điện, LEVEL = 0.

a) Tính toán các tham số để thiết lập chu kỳ và chu kỳ công tác của xung PWM. (0.5 điểm)

- **Thiết lập Timer2**

Chu kỳ xung PWM (hay thời gian Timer2 đếm được) là:

$T_{PWM} = (PR2+1)*(4/F_{OSC})*DIV$

Suy ra,

$T_{PWM(MAX)} = (PR2_{MAX}+1)*(4/F_{OSC})*DIV_{MAX} = (255+1)*(4/8MHz)*16 = 2.048\mu s = 2,048ms$

Để đạt được thời gian T_{PWM} = 2ms cần một giá trị DIV sao cho:

$DIV \geq (16*2ms)/2,048ms = 15,625$

Vì DIV = {1;4;16} nên chọn DIV = 16.

Với DIV = 16, suy ra:

$PR2 = (T_{PWM}*F_{OSC})/(4*DIV)-1 = (2ms*8MHz)/(4*16)-1 = 249$

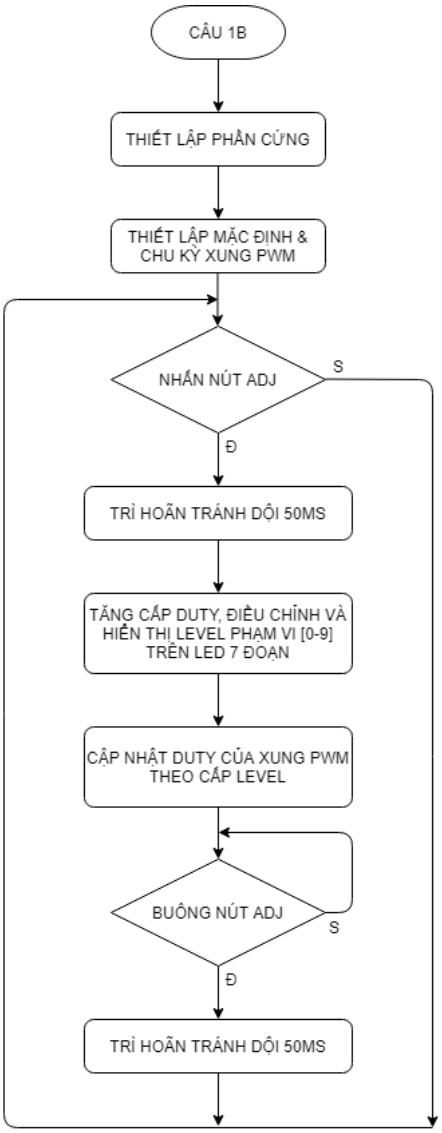
Tóm lại, bộ giá trị để thiết lập cho Timer2 tạo được xung PWM có T_{PWM} = 2ms là {DIV;PR2} = {16;249}.

- **Thiết lập PWM**

$value = (T_{pwm}*Duty*F_{osc})/(100\%*DIV) = (2ms*Duty*8MHz)/(100\%*16)$

- Trường hợp Duty = 10%
  $value = (2ms*10\%*8MHz)/(100\%*16) = (2*10^{-3}*10\%*8*10^{6})/(100\%*16) = 100$
- Trường hợp Duty = 20%
  $value = (2ms*20\%*8MHz)/(100\%*16) = (2*10^{-3}*20\%*8*10^{6})/(100\%*16) = 200$
- Với cách tính tương tự, chúng ta tìm được bộ giá trị để thiết lập chu kỳ công tác DUTY = [0%, 10%, …, 80%, 90%] của xung PWM là:
  value = [0,100,200,300,400,500,600,700,800,900]

b) Vẽ lưu đồ. (1.0 điểm)



c) Viết chương trình C. (1.5 điểm)

```
#include<16F887.h>
#fuses INTRC
#use delay(CLOCK=8MHz)

#define btnADJ       PIN_E0
//      segLEVEL-----PORTD
//                  '0'  '1'  '2'  '3'  '4'  '5'  '6'  '7'  '8'  '9'
char Code7Seg[10]={0xC0,0xF9,0xA4,0xB0,0x99,0x92,0x82,0xF8,0x80,0x90};
unsigned int8 valueLEVEL;

void main()
{
set_tris_d(0b00000000);
set_tris_e(0xFF);

valueLEVEL = 0;
output_d(Code7Seg[valueLEVEL]);

set_tris_c(0b11111001);
//PR2=(Period*Fosc)/(4*DIV)-1=(0.8*1000*8)/(4*16) -1 = 249
```

```
setup_timer_2(T2_DIV_BY_16,249,1);//T=PWM Period = 2ms; DIV=16;PR=249
set_timer2(0);

//value=(Tpwm*Duty*Fosc)/(100%*DIV)=(2ms*Duty*8MHz)/(100%*16)
set_pwm1_duty(100*valueLEVEL);//valueLEVEL = Duty/10
setup_ccp1(CCP_PWM);

while(TRUE)
{
if(input(btnADJ)==0)
{
delay_ms(50);
valueLEVEL++;
if(valueLEVEL==10) valueLEVEL=0;else;
output_d(Code7Seg[valueLEVEL]);
set_pwm1_duty(100*valueLEVEL);
while(input(btnADJ)==0);
delay_ms(50);
}
else;
}
}
```

***Lưu ý**: Mặc dù hàm **`set_pwm1_duty(value)`** có **`value`** có thể là biến hoặc hằng số nhưng hầu hết các phiên bản của PCWH Compiler (CCS) thường hay bị lỗi biên dịch khi **`value`** ở dạng biến số là một biểu thức, chẳng hạn ở đây là **`100*valueLEVEL`**, vì vậy để có kết quả biên dịch đúng, tốt nhất là viết theo cách **`value`** ở dạng hằng số nhưng cách viết sẽ dài hơn. Trong đáp án này, không xét đến cách viết dùng **`value`** như hằng số.*

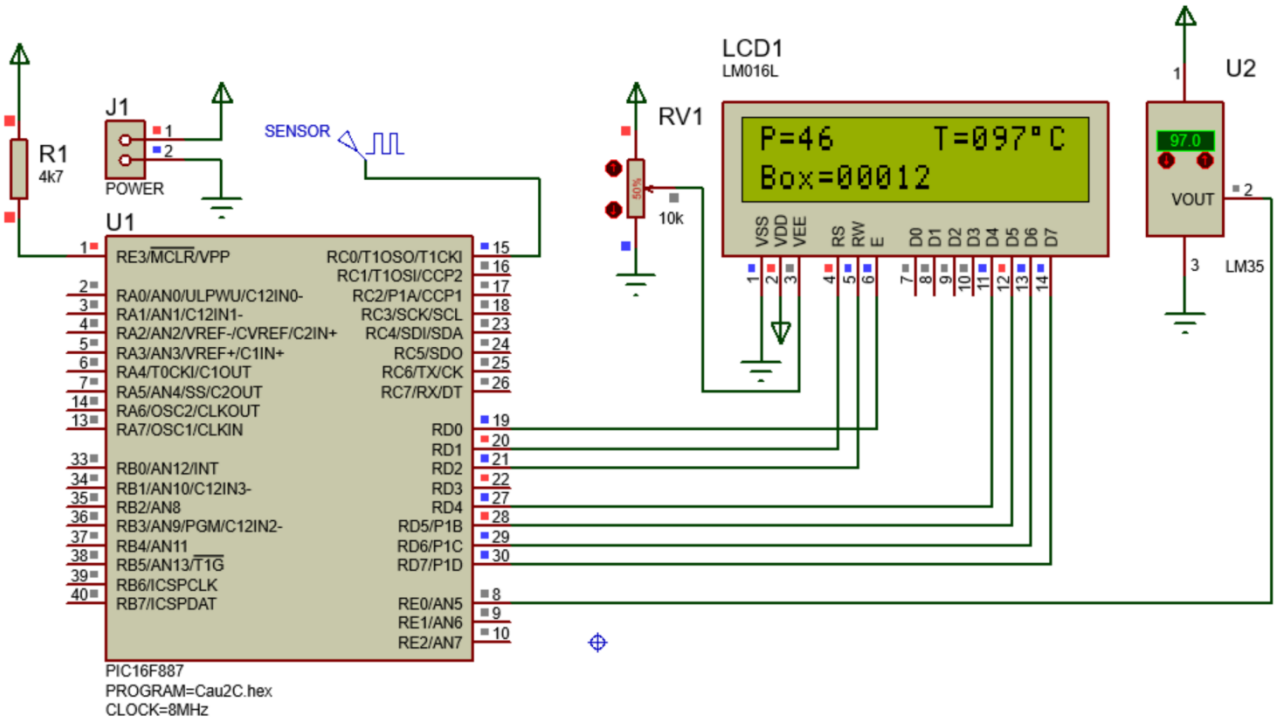
Câu 2: (4 điểm)

Xét một ứng dụng PIC16F887 (tùy chọn tần số và nguồn dao động trong hoặc ngoài) đếm sản phẩm và đo nhiệt độ với các yêu cầu như sau:

- Đếm sản phẩm dùng Counter (tùy chọn Counter sao cho hợp lý). Mỗi hộp có 50 sản phẩm.
- Đo nhiệt độ dùng cảm biến LM35 (được nối với AN5). Giả sử môi trường có nhiệt độ $T \geq 0^oC$.
- Counter chỉ đếm khi nhiệt độ môi trường $T \leq 100^oC$.
- Các giá trị Sản phẩm (P =), số lượng Hộp (Box =) và Nhiệt độ (T =) được hiển thị trên một LCD 16x2 (tùy chọn kết nối PORT). Có định dạng hiển thị như minh họa trên hình:

P = 09	T = 102°C
Box = 00307	

a) Vẽ mạch nguyên lý. (1.0 điểm)



b) Lập công thức tính giá trị nhiệt độ. (0.5 điểm)

- Tính toán độ phân giải

Theo yêu cầu thì các ngõ vào điện áp tham chiếu là

$$V_{REF-} = 0V, V_{REF+} = V_{DD} = 5V$$

Nên độ phân giải (step size) là

$$SS = \frac{V_{REF+} - V_{REF-}}{2^{10} - 1} = \frac{5000mV}{1023} = 4{,}887mV$$

Thông số 2^{10} là do ADC 10 bit.

Điện áp toàn giai full scale = FS:

$$FS = SS \times 1023 = 4{,}887mV \times 1023 = 5{,}000mV = 5V$$

Độ phân giải ADC là 4,887mV không tương thích với độ phân giải của cảm biến LM35 bằng 10mV, tỉ lệ chênh lệch là:

$$\frac{10mV}{4{,}887mV} = 2{,}046$$

Công thức tính giá trị nhiệt độ: **T = (Giá trị đọc từ ADC) : 2,046** **[°C]**

c) Viết chương trình C. (2.5 điểm)

```
#include <16F887.h>
#device ADC=10
#fuses INTRC_IO
#use delay(CLOCK=8MHz)
#include <lcd.c>

unsigned int16 valueP,valueBox,valueT,oldT;

void main()
{
set_tris_c(0xFF);

lcd_init();
set_timer1(0);
```

```
set_tris_e(0xFF);
setup_adc_ports(sAN5|VSS_VDD);
setup_adc(ADC_CLOCK_INTERNAL);
setup_timer_1(T1_DIV_BY_1);
while(TRUE)
{
set_adc_channel(5);
delay_us(20);
valueT = read_adc()/2.046;
if(valueT != oldT)
{
if(valueT>100) setup_timer_1(T1_DISABLED);
else setup_timer_1(T1_EXTERNAL);
oldT = valueT;
}
else;

lcd_gotoxy(10,1);
printf(lcd_putc,"T=%03Lu",valueT);
lcd_putc(223);lcd_putc('C');

valueP = get_timer1();
if(valueP==50+1)
{
valueP=1;set_timer1(1);
valueBox++;
}
else;

lcd_gotoxy(1,1);
printf(lcd_putc,"P=%02Lu",valueP);
lcd_gotoxy(1,2);
printf(lcd_putc,"Box=%05Lu",valueBox);
}
}
```

Câu 3: (3 điểm)

Một ứng dụng kết nối hai bộ vi điều khiển (VĐK) PIC16F887 (tùy chọn tần số và nguồn dao động trong hoặc ngoài) qua UART dùng đường truyền RS232. Giao thức: 9600 bps; 8 bit dữ liệu; 1 STOP; không kiểm tra chẵn lẻ. Yêu cầu cụ thể như sau:

- Trên bộ VĐK1 có nút RUN (nối với RD0), nút STOP (nối với RD1), và một Led đơn (tên gọi STATUS, nối với RD2).
- Trên bộ VĐK2 có một Led đơn (tên gọi BLINK, được nối với RE0).
- Nhấn RUN để nhấp nháy Led BLINK (sáng 1s, tắt 1s, liên tục). Nhấn STOP sẽ tắt Led BLINK.
- Led STATUS sáng khi Led BLINK đang nhấp nháy, và tắt khi Led BLINK tắt. Trạng thái của Led STATUS được hồi tiếp từ VĐK2 về VĐK1.

a) Viết chương trình C cho VĐK1. (1.5 điểm)

```
#include<16F887.h>
#fuses INTRC_IO
#use delay(CLOCK=8MHz)
#use rs232(BAUD=9600,BITS=8,STOP=1,PARITY=N,RCV=PIN_C7,XMIT=PIN_C6)

#define btnRUN    PIN_D0
#define btnSTOP   PIN_D1
#define ledSTATUS PIN_D2

unsigned char dataRx;

void main()
{
set_tris_c(0b10111111);
set_tris_d(0b11111011);
dataRx='0';
while(TRUE)
{
if(kbhit()==1)dataRx=getc();else;

if(dataRx=='1') output_high(ledSTATUS);else
if(dataRx=='0') output_low(ledSTATUS);else;

if(input(btnRUN)==0) putc('R');else
if(input(btnSTOP)==0) putc('S');else;

}
}
```

b) Viết chương trình C cho VĐK2. (1.5 điểm)

```
#include<16F887.h>
#fuses INTRC_IO
#use delay(CLOCK=8MHz)
#use rs232(BAUD=9600,BITS=8,STOP=1,PARITY=N,RCV=PIN_C7,XMIT=PIN_C6)

#define ledBLINK PIN_E0

unsigned char dataRx;
void delay_msRx(unsigned int16 t);
void main()
{
set_tris_c(0b10111111);
set_tris_e(0b11111110);
dataRx='S';
while(TRUE)
{
if(kbhit()==1)dataRx=getc();else;
if(dataRx=='R')
{
output_toggle(ledBLINK);
delay_msRx(1000);
```

```
}
else
if(dataRx=='S') output_low(ledBLINK);
else;
}
}

void delay_msRx(unsigned int16 t)
{
unsigned int16 i;
for(i=0;i<t;i++)
{
if(kbhit()==1)
{
dataRx=getc();
if(dataRx=='R') putc('1');else
if(dataRx=='S') putc('0');else;
}
else;
delay_ms(1);
}
}
```